Lab 7: Fuzzing for Input Validation Bugs

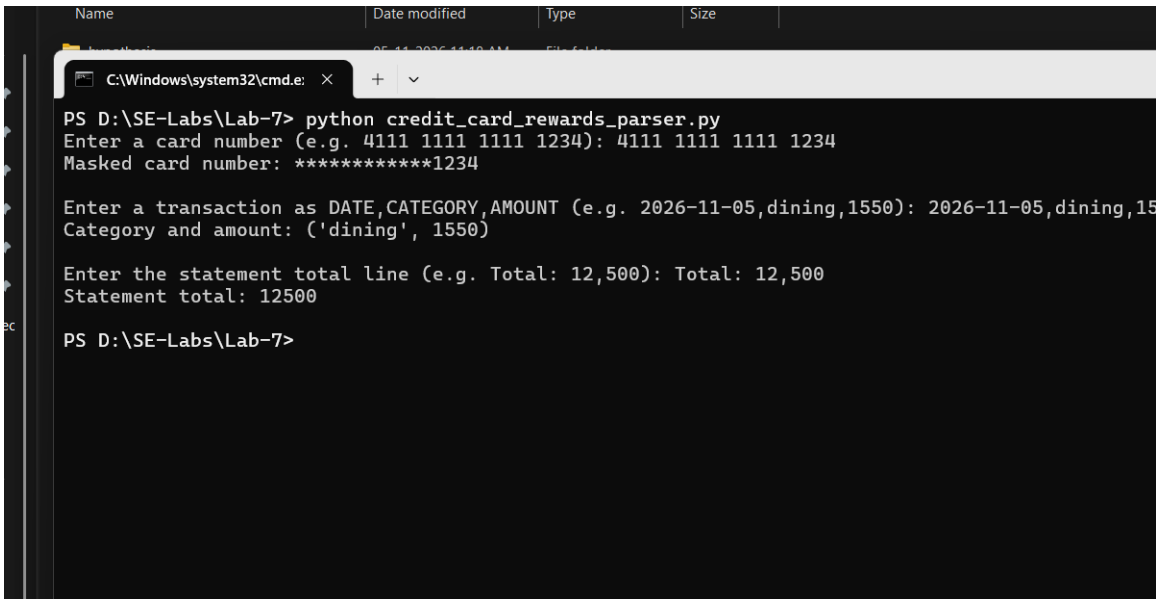
Name: SIDDU SANGANNA SHIRASAGI
SRN: PES1UG25CS846

Objective

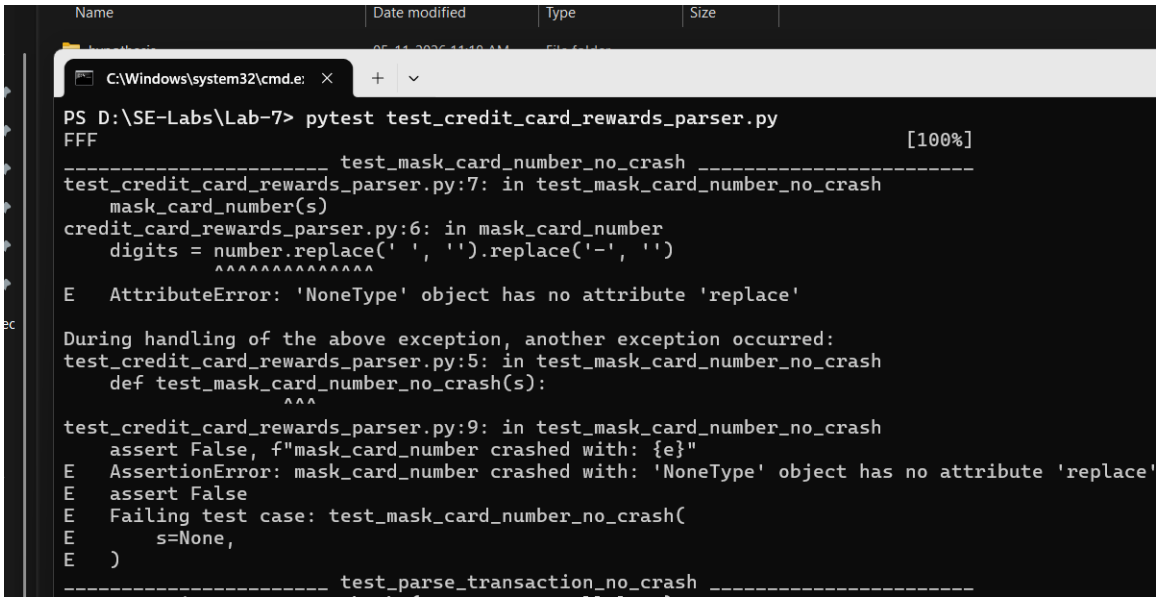
Use the Hypothesis library to perform property-based fuzz testing on credit_card_rewards_parser.py, identify input validation crashes, and harden the code using defensive programming.

Execution Screenshots

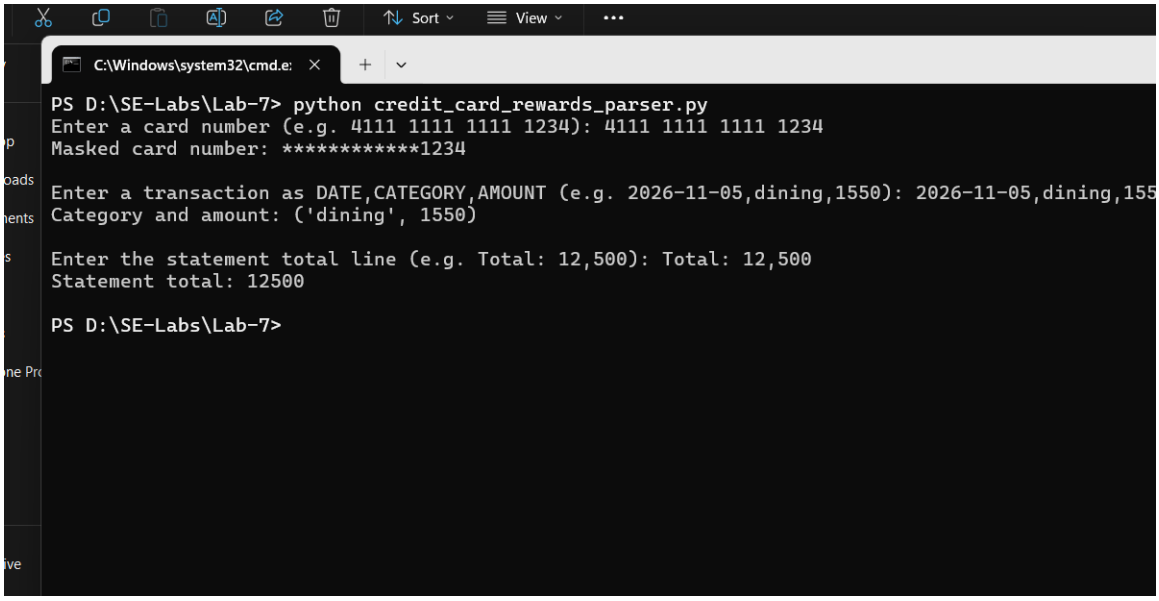
SS1: Running credit_card_rewards_parser.py (Buggy Module)



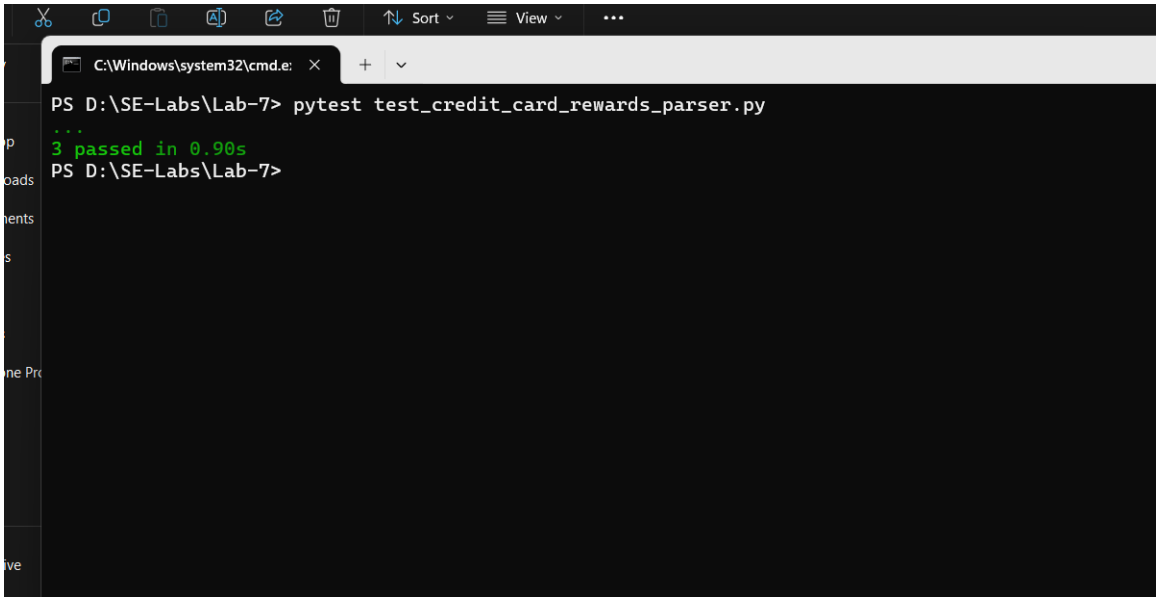
SS2: Running pytest test_credit_card_rewards_parser.py (Failing on Buggy Module)



SS3: Running credit_card_rewards_parser.py (Fixed Module)



SS4: Running pytest test_credit_card_rewards_parser.py (All Tests Passing)



Bugs Discovered by Hypothesis

Function	Assumed Input	Failing Input Found	Error / Exception
mask_card_number	Always a string with card digits	s = None	AttributeError: 'NoneType' has no attribute 'replace'

parse_transaction	Always 'date,category,amount' with int amount	s = None s = '' s = 'date,cat,abc'	AttributeError (on None) IndexError: list index out of range (on '') ValueError: invalid literal for int()
parse_statement_total	Always contains ':' followed by integer	s = None s = '' s = ':'	AttributeError (on None) IndexError: list index out of range ValueError: invalid literal for int()

Reflection Questions

1. What types of bugs does fuzz testing reveal that traditional unit testing usually misses?

Traditional unit tests only check inputs the developer thought of beforehand (the happy path and a few expected edge cases). Fuzz testing generates hundreds of unexpected, malformed, and boundary inputs automatically (such as None, empty strings, missing separators, and bad data types). This easily reveals unhandled crashes like AttributeError, IndexError, and ValueError that manual tests tend to miss.

2. What edge cases and malformed inputs did Hypothesis uncover in each of the three functions?

• mask_card_number: Failed on s = None, causing an AttributeError because NoneType does not have a .replace() method.
• parse_transaction: Failed on s = None (AttributeError on split), empty string s = '' (IndexError because parts has only 1 element), and non-integer amounts like 'abc' (ValueError on int()).
• parse_statement_total: Failed on s = None (AttributeError), empty string or lines without a colon (IndexError on text.split(':')[1]), and strings like ':' with no number after the colon (ValueError).

3. How did you harden the functions against invalid inputs, and what are the trade-offs between defensive parsing vs crashing?

I hardened the functions by adding isinstance(..., str) checks, checking list length and delimiter existence before indexing, and wrapping int() conversion in try-except blocks, returning None or '' when input is invalid.

Trade-offs: Defensive parsing prevents unexpected crashes and keeps the service running even when users provide garbage input. However, returning fallback defaults like None can sometimes hide upstream bugs if the caller forgets to check the result. Raising exceptions makes bugs immediately obvious, but requires callers to add try-except blocks everywhere to prevent the program from crashing.